

Marin's Secrets - Cryptohack

cryptography

Analisis

deksripsi chall:

I've found a super fast way to generate primes from my secret list.

Challenge files:

- [marin.py](#)
- [output.txt](#)

isi source code:

```
#!/usr/bin/env python3

import random
from Crypto.Util.number import bytes_to_long
from secret import secrets, flag

def get_prime(secret):
    prime = 1
    for _ in range(secret):
        prime = prime << 1
    return prime - 1

random.shuffle(secrets)

m = bytes_to_long(flag)
p = get_prime(secrets[0])
q = get_prime(secrets[1])
n = p * q
e = 0x10001
c = pow(m, e, n)

print(f"n = {n}")
print(f"e = {e}")
print(f"c = {c}")
```

isi output:

n:

6584162748301845441250275199214435157898882641560747330992440401262136824
9771403279811639928817650246282925578452597772290301871443430969810820838
8664768262754316426220651576623731617882923164117579624827261244506084274

3712502778493516316794411710184180184980399964725498931505771893028715203
1171517973071431218145624509784849166979599728983061298805852396838480882
2828370900198489249243399165125219244753790779764466236965135793576516193
213175061401667388622283620427170540146790329534410340215068560170810626
17572351195418505899388715709795992029559042119783423597324707100694064675
90923871757305876411889322511160270383808061856540113990214306990111717420
4252871948846864436771808616432457102844534843857198735242005309073939051
4337909467266722346432593495351862685716290779375978388013379730922856087
4420995153319986822804000443213259707339036335789237999765587885769633489
2216345070227646749851381208554044940444182864026513709449823489593439017
3663588696481682387350875938083444843651362842197252338116053318150074245
8289082188726068288663254361310925286211432637207778536929257090059481448
1097443781269562647303671428895764224084402259605109600363098950091998891
3758128395236132956672538139784348791727812172856528954691941812183430787
5450169474659873821524376974795657255598959459818063909834489117587945599
4652382137038240166358066403475457

e: 65537

c:

4002804630889304323192803591151949775825173636105324642952106695304078707
5343912745540138456970542562144594399296338098308491738542863122304690883
7804126399345875252917090184158440305503817193246288672986488987883177380
3073770250792660302626509325752051418534133025584603642423555312729674814
0941478363455879117582781654076754594453423818907903019284328859693497969
3517964655661507346729751987928147021620165009965051933278913952899114253
3010447475873108304191906232825789315895875045550053615715725619168660634
5881296531447416049906752506749514015009211962092836300746739092013071752
1169105167963364154636472055084012592138570354390246779276003156184676298
71074658310470051646609103451076502716795611786905193811645737038473744096
5109619578227422049806566060571831017610877072484262724789571076529586427
40578012109654694281232232480714513701794226686353498908211518906556001184
1150908380937354301243153206428896320576609904361937035263985348984794208
1988926158989070059554035294708471242695123161917539502037945786560293245
0668829344657159850604219821908032574732863623204093676178855842152896027
98328021275621158523049468676283165029595622744854838674817311493382090097
5322946392485593010327119783137098248870345646338591480124682866221262200
6947380115549529820197355738525329885232170215757585685484402344437894981
555179129287164971002033759724456

di source code kita dikasih tau bahwa p dan q di buat menggunakan fungsi
get_prime().dimana bisa kita lihat bahwa di fungsi get_prime() bilangan prima untuk p dan q
di cari menggunakan metode marsenne.

rumus untuk mencari bilangan prima nya adalah dengan $(2^n) - 1$ dimana n adalah bilangan
prima dari metode mersenne.

Solusi

jadi untuk mencari nilai dari p dan q maka kita bisa langsung memasukan nilai dari n ke factordb.com atau kita bisa membrute force nya. untuk membrute force nya pertama kita harus mencari bilangan prima yang ada di metode mersenne dari yang terkecil sampai minimal sebesar bit dari flag. setelah itu kita coba semua nilai dari bilangan prima itu dengan mem bitwise nya lalu di kurang i 1

solver:

```
from Crypto.Util.number import bytes_to_long, isPrime, inverse, long_to_bytes
maresene_prime= [2, 3, 5, 7, 13, 17, 19, 31, 61, 89, 107, 127, 521, 607, 1279, 2203, 2281, 3217, 4253, 4423, 9689, 9941, 11213]
n =
65841627483018454412502751992144351578988826415607473309924404012621368
24977140327981163992881765024628292557845259777229030187144343096981082
08388664768262754316426220651576623731617882923164117579624827261244506
08427437125027784935163167944117101841801849803999647254989315057718930
28715203117151797307143121814562450978484916697959972898306129880585239
68384808822828370900198489249243399165125219244753790779764466236965135
79357651619321317506140166738862222836204271705401467903295344103402150
68560170810626175723511954185058993887157097959920295590421197834235973
24707100694064675909238717573058764118893225111602703838080618565401139
90214306990111717420425287194884686443677180861643245710284453484385719
87352420053090739390514337909467266722346432593495351862685716290779375
97838801337973092285608744209951533199868228040004432132597073390363357
89237999765587885769633489221634507022764674985138120855404494044418286
40265137094498234895934390173663588696481682387350875938083444843651362
84219725233811605331815007424582890821887260682886632543613109252862114
32637207778536929257090059481448109744378126956264730367142889576422408
44022596051096003630989500919988913758128395236132956672538139784348791
72781217285652895469194181218343078754501694746598738215243769747956572
55598959459818063909834489117587945599465238213703824016635806640347545
7
found = False
for i in range(len(maresene_prime)):
    for j in range(i, len(maresene_prime)):
        a = maresene_prime[i]
        b = maresene_prime[j]
        print(f"a = {a}")
        print(f"b = {b}\n")
        p = (1 << a) - 1
        q = (1 << b) - 1
```

```
        if p*q == n:
            print(f"nilai p = {p}")
            print(f"nilai q = {q}")
            found = True
            break

    if found:
        break

p =
14759799152141802350848986227373817363120661453331697751477712164785702
97878078949377407337049389289382748507531496480477281264838760259191814
46336533026954049696120111343015690239609398909022625932693502528140961
49834993882228314485986018343185362309237726413902094902318364468996082
10795482963763094236630945410832793769905399982457186322944729636418890
62337217172374210563644036821845964963294853869690587265048691443463745
75072804418236768135178520993486608471725794084223166780976702240119902
80170474894487426924742108823536808485072502240519452587542875349976558
57267022963396257521263747789778550155264652260998886991401354048380986
5681250419497686697771007

q =
44608755718375842957115170640210180988620863241285990111199121996340468
57928204733691125452690039890261532459311243167023957587056936793647909
03497461147071065254193353938124978226307947312410798874869040070279328
42881031175484410809487825249486676096958699812898264587759602897917153
69625030684296173317021847503245830091718321049160501576288866063721455
01702225925125224076829605427173573964812995250569412480720738476855293
68166671284483119087762060678666386219024011857073683190188647922581041
47140789353865624979681787291276295949244119609613867139462798992750069
54917139758796061223803393537381034666494402951052059047968693255388647
930440925104186817009640171764133172418132836351

c
=4002804630889304323192803591151949775825173636105324642952106695304078
70753439127455401384569705425621445943992963380983084917385428631223046
90883780412639934587525291709018415844030550381719324628867298648898788
31773803073770250792660302626509325752051418534133025584603642423555312
72967481409414783634558791175827816540767545944534238189079030192843288
59693497969351796465566150734672975198792814702162016500996505193327891
39528991142533010447475873108304191906232825789315895875045550053615715
72561916866063458812965314474160499067525067495140150092119620928363007
46739092013071752116910516796336415463647205508401259213857035439024677
```

```

92760031561846762987107465831047005164660910345107650271679561178690519
38116457370384737440965109619578227422049806566060571831017610877072484
26272478957107652958642740578012109654694281232232480714513701794226686
35349890821151890655600118411509083809373543012431532064288963205766099
04361937035263985348984794208198892615898907005955403529470847124269512
31619175395020379457865602932450668829344657159850604219821908032574732
86362320409367617885584215289602798328021275621158523049468676283165029
59562274485483867481731149338209009753229463924855930103271197831370982
48870345646338591480124682866221262200694738011554952982019735573852532
98852321702157575856854844023444378949815551791292871649710020337597244
56
e = 65537

phi = (p-1)*(q-1)
d = pow(e,-1,phi)

m = pow(c,d,n)
print(long_to_bytes(m))
flag = b'crypto{Th3se_Pr1m3s_4r3_t00_r4r3}'
print(flag.hex())

print()

```

atau bisa kayak gini:

```

from Crypto.Util.number import isPrime, long_to_bytes

# Output dari challenge
n =
65841627483018454412502751992144351578988826415607473309924404012621368
24977140327981163992881765024628292557845259777229030187144343096981082
08388664768262754316426220651576623731617882923164117579624827261244506
08427437125027784935163167944117101841801849803999647254989315057718930
28715203117151797307143121814562450978484916697959972898306129880585239
68384808822828370900198489249243399165125219244753790779764466236965135
79357651619321317506140166738862222836204271705401467903295344103402150
68560170810626175723511954185058993887157097959920295590421197834235973
24707100694064675909238717573058764118893225111602703838080618565401139
90214306990111717420425287194884686443677180861643245710284453484385719
87352420053090739390514337909467266722346432593495351862685716290779375
97838801337973092285608744209951533199868228040004432132597073390363357
89237999765587885769633489221634507022764674985138120855404494044418286
40265137094498234895934390173663588696481682387350875938083444843651362
84219725233811605331815007424582890821887260682886632543613109252862114

```

```

32637207778536929257090059481448109744378126956264730367142889576422408
44022596051096003630989500919988913758128395236132956672538139784348791
72781217285652895469194181218343078754501694746598738215243769747956572
55598959459818063909834489117587945599465238213703824016635806640347545
7
e = 65537
c =
40028046308893043231928035911519497758251736361053246429521066953040787
07534391274554013845697054256214459439929633809830849173854286312230469
08837804126399345875252917090184158440305503817193246288672986488987883
17738030737702507926603026265093257520514185341330255846036424235553127
29674814094147836345587911758278165407675459445342381890790301928432885
96934979693517964655661507346729751987928147021620165009965051933278913
95289911425330104474758731083041919062328257893158958750455500536157157
25619168660634588129653144741604990675250674951401500921196209283630074
67390920130717521169105167963364154636472055084012592138570354390246779
27600315618467629871074658310470051646609103451076502716795611786905193
81164573703847374409651096195782274220498065660605718310176108770724842
62724789571076529586427405780121096546942812322324807145137017942266863
53498908211518906556001184115090838093735430124315320642889632057660990
43619370352639853489847942081988926158989070059554035294708471242695123
16191753950203794578656029324506688293446571598506042198219080325747328
63623204093676178855842152896027983280212756211585230494686762831650295
95622744854838674817311493382090097532294639248559301032711978313709824
88703456463385914801246828662212622006947380115549529820197355738525329
88523217021575758568548440234443789498155517912928716497100203375972445
6

p = None
q = None

# 1. Cari nilai p dengan menguji eksponen prima a
# Panjang bit n sekitar 4000-bit, jadi eksponen a sekitar 1000 - 3000
for a in range(2, 4000):
    if isPrime(a):
        candidate_p = (1 << a) - 1
        if n % candidate_p == 0:
            p = candidate_p
            q = n // candidate_p
            print(f"[+] Faktor ditemukan!")
            print(f"[+] Eksponen a = {a}")
            break

```

```
# 2. Dekripsi RSA
phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)
m = pow(c, d, n)

# 3. Print Flag
flag = long_to_bytes(m)
print(f"[+] Flag: {flag.decode()}")
```